

سوالات و پاسخ‌های امتحان میان‌ترم ساختمان داده

پاسخ‌نامه تشریحی

دانشگاه شهید مدنی آذربایجان

سوال ۱

پیچیدگی قطعه کد زیر را محاسبه کنید.

```
for ( i=1, j=n; i <= j; i++, j- ) cout << i+j;
```

پاسخ:

حلقه for دو متغیر i و j دارد که از دو انتهای بازه $[1, n]$ شروع به حرکت می‌کنند. در هر تکرار، i یک واحد افزایش و j یک واحد کاهش می‌یابد؛ بنابراین فاصله بین این دو در هر تکرار ۲ واحد کم می‌شود. حلقه تا زمانی که $i \leq j$ باشد ادامه دارد، لذا دقیقاً حدود $\frac{n}{2}$ بار اجرا می‌شود. از آنجا که عملیات داخل حلقه زمان ثابت $O(1)$ می‌برد، پیچیدگی زمانی کل قطعه کد برابر با $\Theta(n)$ است.

سوال ۲

پیچیدگی $T(n)$ الگوریتم ترتیبی غیر بازگشتی مرتب‌سازی درجی داده شده را محاسبه کرده و سپس مشخص کنید که $T(n) \in O(?)$, $T(n) \in \Omega(?)$, $T(n) \in \Theta(?)$. همچنین با استفاده از این الگوریتم عناصر ۶۵، ۳، ۱، ۲ را مرتب کنید.

پاسخ:

الف) پیچیدگی الگوریتم مرتب‌سازی درجی (Insertion Sort):

- بهترین حالت: زمانی رخ می‌دهد که آرایه از قبل مرتب باشد. حلقه داخلی در همان گام اول به دلیل شرط $a[i] > key$ متوقف می‌شود و تنها حلقه بیرونی $n - 1$ بار اجرا می‌شود. بنابراین $T(n) \in \Omega(n)$.
- بدترین حالت: زمانی رخ می‌دهد که آرایه به صورت معکوس مرتب باشد. حلقه داخلی برای هر k ، دقیقاً k بار اجرا می‌شود. مجموع این دفعات $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$ خواهد بود که از مرتبه n^2 است. بنابراین $T(n) \in O(n^2)$.
- کران Theta: به طور کلی برای همه حالات یک کران Θ مشترک وجود ندارد، اما در بدترین حالت، پیچیدگی $\Theta(n^2)$ و در بهترین حالت $\Theta(n)$ است.

ب) ردیابی الگوریتم برای آرایه $A = [65, 3, 1, 2]$:

- آرایه اولیه: $[65, 3, 1, 2]$
- تکرار $k=1$: $key = 3$. عنصر ۶۵ بزرگتر از ۳ است، پس شیفت داده می‌شود. آرایه می‌شود: $[3, 65, 1, 2]$
- تکرار $k=2$: $key = 1$. عناصر ۶۵ و ۳ هر دو از ۱ بزرگتر هستند، پس به راست شیفت داده می‌شوند. آرایه می‌شود: $[1, 3, 65, 2]$
- تکرار $k=3$: $key = 2$. عنصر ۶۵ و ۳ از ۲ بزرگتر هستند، پس شیفت می‌خورند. عنصر ۱ کوچکتر از ۲ است، پس شیفت متوقف می‌شود. جایگذاری ۲ انجام شده و آرایه می‌شود: $[1, 2, 3, 65]$

سوال ۳

پیچیدگی $T(n)$ الگوریتم بازگشتی برج هانوی داده شده را پیدا کرده و سپس خروجی این الگوریتم بازگشتی را به ازای ۴ پیدا کنید.

پاسخ:

الف) محاسبه پیچیدگی زمانی: رابطه بازگشتی الگوریتم برج هانوی به این شکل است: $T(n) = 2T(n-1) + 1$. با باز کردن این رابطه یا استفاده از روش های حل، به جواب $T(n) = 2^n - 1$ می‌رسیم. بنابراین پیچیدگی زمانی الگوریتم از نوع نمایی و برابر با $\Theta(2^n)$ است.

ب) خروجی به ازای $n = 4$ (با فراخوانی $\text{Hanoi}(A, B, C, 4)$): از آنجا که $n = 4$ ، الگوریتم به $2^4 - 1 = 15$ حرکت نیاز دارد. خروجی چاپی دقیق تابع به شکل زیر خواهد بود:

- | | | |
|-----------------------|-----------------------|----------------------|
| 11. $C \rightarrow A$ | 6. $C \rightarrow B$ | 1. $A \rightarrow B$ |
| 12. $B \rightarrow C$ | 7. $A \rightarrow B$ | 2. $A \rightarrow C$ |
| 13. $A \rightarrow B$ | 8. $A \rightarrow C$ | 3. $B \rightarrow C$ |
| 14. $A \rightarrow C$ | 9. $B \rightarrow C$ | 4. $A \rightarrow B$ |
| 15. $B \rightarrow C$ | 10. $B \rightarrow A$ | 5. $C \rightarrow A$ |

سوال ۴

الگوریتم بازگشتی ارائه نمایید که با تعداد مقایسه (پیچیدگی) $2 - \frac{3n}{2}$ بین n عدد، کوچکترین و بزرگترین آنها را پیدا کند.

پاسخ:

برای رسیدن به تعداد دقیق $2 - \frac{3n}{2}$ مقایسه، باید از روش تقسیم و غلبه (Divide and Conquer) استفاده کرد. الگوریتم بازگشتی به زبان C++ به این صورت است:

```
void findMinMax(int A[], int low, int high, int &min, int &max) {
    if (low == high) {
        min = max = A[low];
        return;
    }
    if (high == low + 1) {
        if (A[low] < A[high]) {
            min = A[low]; max = A[high];
        } else {
            min = A[high]; max = A[low];
        }
        return;
    }
}
```

```

int mid = (low + high) / 2;
int min1, max1, min2, max2;
findMinMax(A, low, mid, min1, max1);
findMinMax(A, mid + 1, high, min2, max2);
min = (min1 < min2) ? min1 : min2;
max = (max1 > max2) ? max1 : max2;
}

```

سوال ۵

رابطه بازگشتی زیر را با استفاده از روش درخت بازگشت حل کنید.

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(\frac{n}{2}) + \log n & n \geq 2 \end{cases}$$

پاسخ:

با رسم درخت بازگشت، ساختار هزینه‌ها در سطوح مختلف را بررسی می‌کنیم:

- سطح ۰ (ریشه): دارای هزینه $\log n$ است.
- سطح ۱: دارای ۲ گره است که اندازه هر کدام $n/2$ است. مجموع هزینه: $2 \log(n/2) = 2(\log n - 1)$.
- سطح i : دارای 2^i گره با اندازه ورودی $n/2^i$ است. هزینه این سطح: $2^i \log(n/2^i) = 2^i(\log n - i)$.
- ارتفاع درخت: برابر با $k = \log_2 n$ است. در سطح نهایی (برگ‌ها)، n گره با هزینه ثابت $\Theta(1)$ داریم، پس مجموع هزینه برگ‌ها $\Theta(n)$ است.

کل هزینه سیستم حاصل جمع سطوح تا قبل از برگ‌ها به اضافه هزینه برگ‌ها است:

$$T(n) = \sum_{i=0}^{k-1} 2^i (\log n - i) + \Theta(n)$$

$$T(n) = \log n \left(\sum_{i=0}^{k-1} 2^i \right) - \left(\sum_{i=0}^{k-1} i 2^i \right) + \Theta(n)$$

با حل سری هندسی $\sum_{i=0}^{k-1} 2^i = 2^k - 1 = n - 1$ و سری حسابی-هندسی $\sum_{i=0}^{k-1} i 2^i = (k-2)2^k + 2 = (n-2)n + 2$ نتیجه ساده می‌شود:

$$T(n) = \log n(n-1) - (n \log n - 2n + 2) + n$$

$$T(n) = n \log n - \log n - n \log n + 2n - 2 + n = 3n - \log n - 2$$

در نتیجه جمله غالب $3n$ بوده و پیچیدگی زمانی الگوریتم برابر با $\Theta(n)$ است.

سوال ۶

رابطه بازگشتی $T(n) = 2T(\frac{n}{2}) + n^2$ را با استفاده از روش قضیه اصلی حل کنید.

پاسخ:

طبق قضیه اصلی (Master Theorem) به فرم $T(n) = aT(\frac{n}{b}) + f(n)$ ، مقادیر برابرند با:

$$a = 2, \quad b = 2, \quad f(n) = n^2$$

مقدار معیار ارزیابی برابر است با $n^{\log_b a} = n^{\log_2 2} = n^1 = n$ از آنجایی که $f(n) = n^2$ به صورت چندجمله‌ای از n بزرگتر است، یعنی $f(n) = \Omega(n^{1+\epsilon})$ (با $\epsilon = 1 > 0$)، در **حالت سوم** قضیه اصلی قرار می‌گیریم. حال باید شرط منظم بودن (Regularity Condition) را بررسی کنیم. باید ثابتی مانند $c < 1$ وجود داشته باشد که $af(\frac{n}{b}) \leq cf(n)$ برقرار باشد:

$$2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} \leq c \cdot n^2$$

این نامساوی به ازای $c = \frac{1}{2}$ همواره برقرار است و شرط منظم بودن صدق می‌کند. در نتیجه، پیچیدگی الگوریتم برابر است با:

$$T(n) = \Theta(f(n)) = \Theta(n^2)$$

سوال ۷

آیا $\mathcal{O}(2^n) = 2^{n+1}$ با استفاده از تعریف درست است؟

پاسخ:

طبق تعریف نماد Big-O، عبارت $f(n) \in \mathcal{O}(g(n))$ در صورتی درست است که ثابت‌های مثبت c و n_0 وجود داشته باشند، به طوری که به ازای تمام مقادیر $n \geq n_0$ داشته باشیم:

$$f(n) \leq c \cdot g(n)$$

در این سوال $f(n) = 2^{n+1}$ و $g(n) = 2^n$ است. می‌دانیم که می‌توانیم $f(n)$ را به شکل زیر بازنویسی کنیم:

$$2^{n+1} = 2 \times 2^n$$

اگر ثابت‌ها را به صورت $c = 2$ و $n_0 = 1$ در نظر بگیریم، رابطه زیر به دست می‌آید:

$$2 \times 2^n \leq 2 \times 2^n$$

که کاملاً برقرار است. بنابراین این گزاره **درست** است.